NETWORK INTRUSION DETECTION MODEL

1. Introduction

Network Intrusion Detection is used to identify suspicious or unauthorized activities in network traffic. In this project, a Machine Learning model is developed to classify network connections into two categories: **Normal** and **Attack**.

The **Random Forest Classifier** is used for classification because it works effectively with structured data and can handle a large number of network features.

2. Objective

The main objective of this project is to develop a machine learning model that can analyze network traffic and identify whether a network connection is normal or represents an intrusion.

3. Dataset

The **NSL-KDD dataset** is used in this project. It contains different features related to network connections along with their corresponding attack information.

Two separate datasets are used for training and testing.

Dataset	Number of Records
KDDTrain+	125,973
KDDTest+	22,544

The dataset initially contains **43 columns**, including network features, attack labels, and difficulty information.

4. Data Preprocessing

Before training the model, the dataset was prepared for machine learning. The original attack labels were converted into binary classes:

Label	Class
0	Normal Traffic
1	Attack Traffic

Categorical features such as **protocol_type, service, and flag** were converted into numerical values using Label Encoding.

The **attack** and **difficulty** columns were removed after preparing the target labels.

5. Code Implementation

The following Python code was used for dataset downloading, preprocessing, model training, prediction, evaluation, and saving the trained model.

```
import pandas as pd
import os
import urllib.request
import matplotlib.pyplot as plt
import joblib
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

train_url = "https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain%2B.txt"
test_url = "https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTest%2B.txt"
train_file = "KDDTrain+.txt"
test_file = "KDDTest+.txt"

if not os.path.exists(train_file):
    urllib.request.urlretrieve(train_url, train_file)
if not os.path.exists(test_file):
    urllib.request.urlretrieve(test_url, test_file)

columns = [
'duration','protocol_type','service','flag','src_bytes','dst_bytes','land',
'wrong_fragment','urgent','hot','num_failed_logins','logged_in','num_compromised',
'root_shell','su_attempted','num_root','num_file_creations','num_shells',
'num_access_files','num_outbound_cmds','is_host_login','is_guest_login','count',
'srv_count','serror_rate','srv_serror_rate','rerror_rate','srv_rerror_rate',
'same_srv_rate','diff_srv_rate','srv_diff_host_rate','dst_host_count',
'dst_host_srv_count','dst_host_same_srv_rate','dst_host_diff_srv_rate',
'dst_host_same_src_port_rate','dst_host_srv_diff_host_rate','dst_host_serror_rate',
'dst_host_srv_serror_rate','dst_host_rerror_rate','dst_host_srv_rerror_rate',
'attack','difficulty'
]

train = pd.read_csv(train_file, names=columns)
test = pd.read_csv(test_file, names=columns)

print("Training data:", train.shape)
print("Testing data:", test.shape)

train['label'] = train['attack'].apply(lambda x: 0 if x == 'normal' else 1)
test['label'] = test['attack'].apply(lambda x: 0 if x == 'normal' else 1)
train.drop(['attack','difficulty'], axis=1, inplace=True)
test.drop(['attack','difficulty'], axis=1, inplace=True)

features = pd.concat([train.drop('label',axis=1), test.drop('label',axis=1)],
                     ignore_index=True)
encoders = {}
for col in ['protocol_type','service','flag']:
    le = LabelEncoder()
    features[col] = le.fit_transform(features[col])
    encoders[col] = le

X_train = features.iloc[:len(train)]
X_test = features.iloc[len(train):]
y_train = train['label']
y_test = test['label']

model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("\nAccuracy:", round(accuracy_score(y_test, predictions)*100, 2), "%")
print("\nClassification Report:")
print(classification_report(y_test, predictions, target_names=['Normal','Attack']))

cm = confusion_matrix(y_test, predictions)
ConfusionMatrixDisplay(cm, display_labels=['Normal','Attack']).plot()
plt.title("Network Intrusion Detection")
plt.show()

importance = pd.Series(model.feature_importances_, index=X_train.columns)
importance.nlargest(10).sort_values().plot(kind='barh')
plt.title("Top 10 Important Features")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()

result = pd.DataFrame({
    'Actual': y_test.iloc[:20].map({0:'Normal',1:'Attack'}).values,
    'Predicted': pd.Series(predictions[:20]).map({0:'Normal',1:'Attack'}).values
})
print("\nSample Predictions:")
print(result)

joblib.dump(model, "intrusion_detection_model.pkl")
joblib.dump(encoders, "encoders.pkl")
print("\nModel saved successfully.")
```

6. Machine Learning Model

A **Random Forest Classifier** was used to build the Network Intrusion Detection model.

The model was configured with **100 decision trees (n_estimators = 100)** and a random state of **42**. The KDDTrain+ dataset was used for training, while the KDDTest+ dataset was used for testing.

7. Model Evaluation

After training, the model was evaluated on **22,544 testing records**. The model achieved an overall accuracy of **77.07%**.

Classification Report

Class	Precision	Recall	F1-Score	Support
Normal	0.66	0.97	0.78	9,711
Attack	0.97	0.62	0.75	12,833
Accuracy	-	-	0.77	22,544
Macro Average	0.81	0.80	0.77	22,544
Weighted Average	0.83	0.77	0.77	22,544

The results show that the model performs well in identifying normal traffic, while some attack records are classified as normal.

8. Confusion Matrix

The Confusion Matrix shows the number of correct and incorrect predictions made by the model.

Actual / Predicted	Normal	Attack
Normal	9,434	277
Attack	4,893	7,940

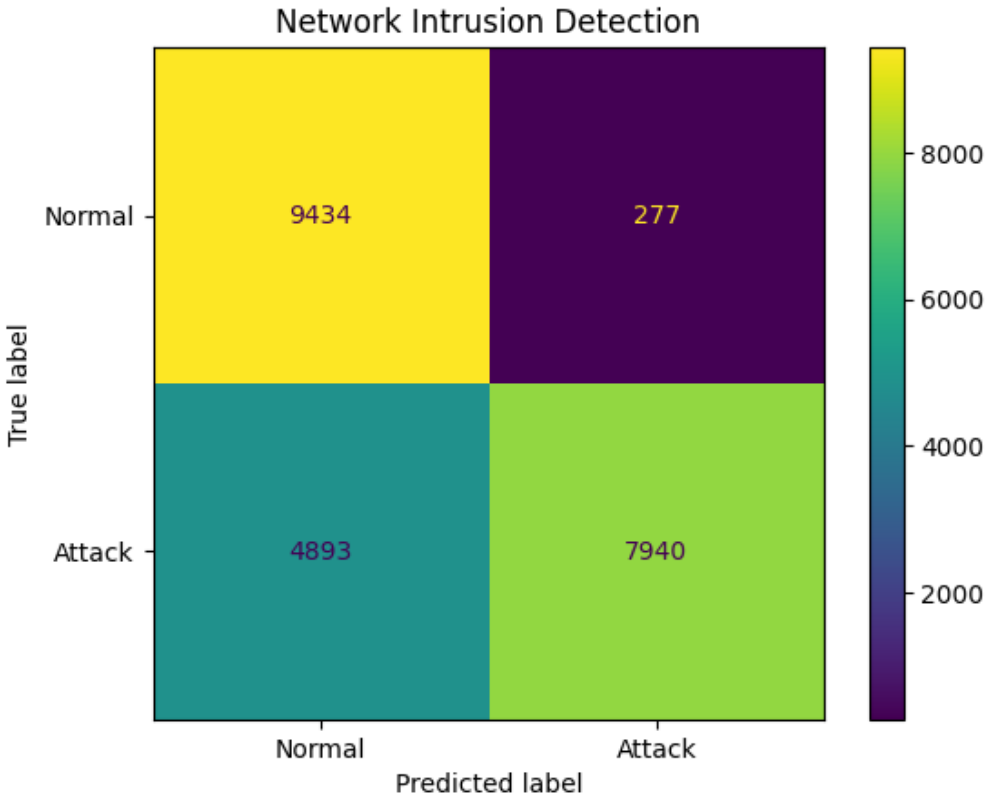

Figure 1: Confusion Matrix of Network Intrusion Detection Model

The confusion matrix shows that **9,434 normal connections** and **7,940 attack connections** were correctly classified. A total of **277 normal connections** were predicted as attacks, while **4,893 attack connections** were predicted as normal.

9. Sample Prediction Output

Some sample predictions obtained from the trained model are shown below.

Samples 1-10

	1	2	3	4	5	6	7	8	9	10
Actual	Attack	Attack	Normal	Attack	Attack	Normal	Normal	Attack	Normal	Attack
Predicted	Attack	Attack	Normal	Attack	Normal	Normal	Normal	Normal	Normal	Normal

Samples 11-20

	11	12	13	14	15	16	17	18	19	20
Actual	Attack	Normal	Attack	Attack	Normal	Normal	Normal	Normal	Normal	Attack
Predicted	Normal	Normal	Attack	Attack	Normal	Normal	Normal	Normal	Normal	Attack

The sample output provides a direct comparison between the actual network traffic class and the class predicted by the model.

10. Limitations

• The model is trained on the NSL-KDD dataset, which may not represent all modern network attacks.

• The model performs only binary classification between Normal and Attack traffic.

• Its performance may vary when applied to real-time network traffic.

11. Future Scope

• Newer network intrusion datasets can be used to improve the model.

• The model can be extended to classify different types of network attacks separately.

• It can be integrated with real-time network traffic monitoring systems.

12. Conclusion

The Network Intrusion Detection model was successfully developed using the **Random Forest Classifier** and the **NSL-KDD dataset**.

The model achieved **77.07% accuracy** on **22,544 testing records** and successfully classified network traffic into Normal and Attack categories. The project demonstrates a simple application of Machine Learning for detecting suspicious activities in network traffic.